📊 Data Types & Distributions

Normal Distribution • All Chart Types • Python Code

Beginner-Friendly Guide for Data Science & Machine Learning

📋 What This Chapter Covers

✅Types of Data — Categorical vs Numerical (and sub-types)
✅All major Probability Distributions explained in simple words
✅Normal / Gaussian Distribution — the MOST important one in ML
✅The 68-95-99.7 Rule — from your images
✅All chart types — Histogram, Box Plot, Bar Chart, and more
✅Discrete vs Continuous distributions
✅How to detect if data is normally distributed
✅How to normalize data — methods and Python code
✅Complete Python code for every concept

Quick Overview — Distribution Family Map

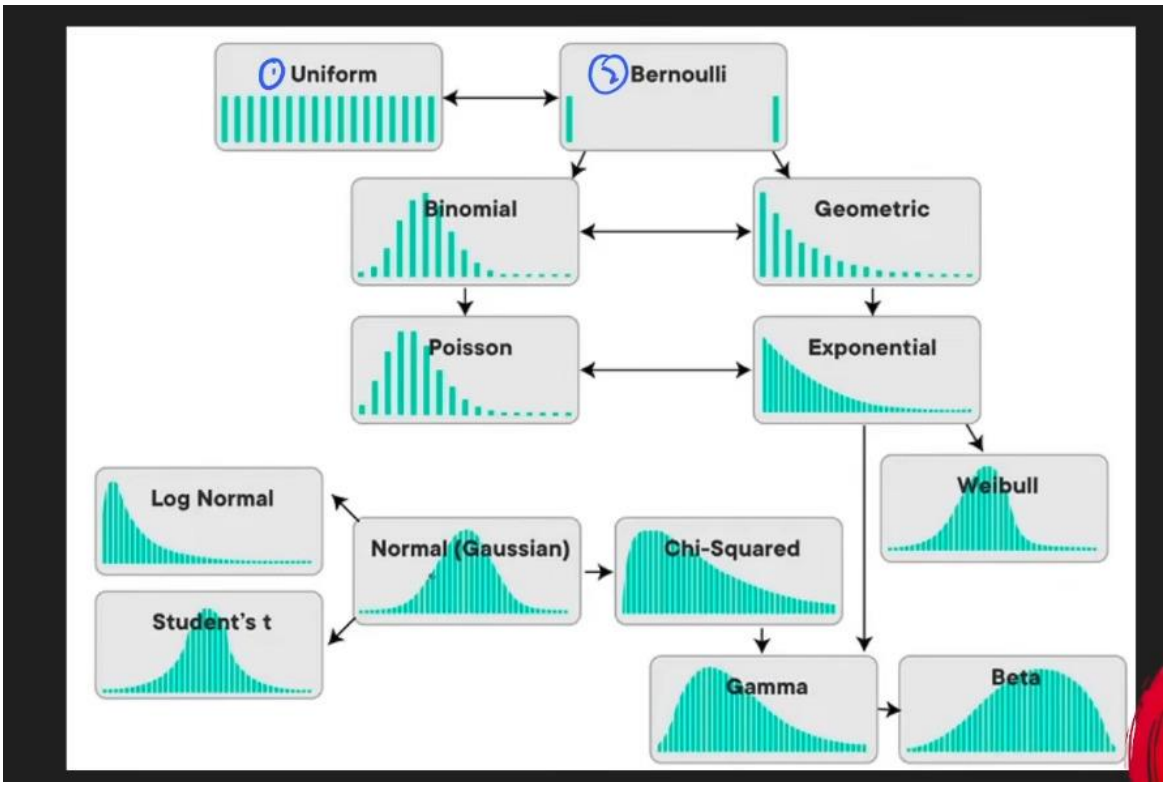

Figure 1: How all probability distributions relate to each other

1 Types of Data

Why Data Types Matter

Before you can choose a chart, distribution, or analysis method,
you MUST know what TYPE of data you are working with.
The wrong type assumption = wrong chart = wrong model = wrong conclusions.

Categorical Data — Data with Labels or Groups

Categorical data represents qualities or labels — things you can PUT INTO GROUPS.

You cannot do math on these (you cannot add 'Red' + 'Blue').

Sub-type	What it is	Examples	Python dtype
Nominal	Labels with NO order — one is not 'more' than another	Blood type (A/B/O), Colours, Country, Gender	object / str / pd.Categorical
Ordinal	Labels WITH order — but gaps between ranks are not equal	Star ratings (1★5★), Education level, Survey scale (Bad/OK/Good)	pd.Categorical(ordered=True)

Nominal vs Ordinal — Easy Trick

Nominal = No order. Red is not 'more' than Blue.
Ordinal = Has order. 5 stars IS more than 1 star. But is 5★FIVE TIMES better than 1★? Not necessarily.

The key: ordinal has RANK but NOT equal spacing between ranks.

Numerical Data — Data with Actual Numbers

Numerical data represents measurements or counts. You CAN do math on these.

Sub-type	What it is	Examples	Key property
Discrete	Countable whole numbers — you CANNOT have half a value	Number of students, Number of goals, Coin flips	Finite or countably infinite gaps between values
Continuous	Any value in a range — infinite	Height (1.731 m), Temperature, Weight, Time	Can take any decimal value — no gaps

	possibilities between two points		
Interval	Numbers WITH equal spacing but NO true zero point	Temperature in °C (0°C is NOT 'no temperature'), Years	Can add/subtract, CANNOT multiply/divide meaningfully
Ratio	Numbers WITH equal spacing AND a true zero point	Height, Weight, Speed, Income (0 = truly nothing)	Full math allowed — ratios make sense (2x as tall)

Interval vs Ratio — The Zero Point Test

Ask: Does zero mean TRULY NOTHING?

Temperature 0°C = NOT 'no temperature'. It's just freezing point. → INTERVAL
Height 0 cm = truly no height. Zero means nothing. → RATIO
Income Rs. 0 = truly no income. → RATIO

Only RATIO lets you say '2x as much'. You cannot say 20°C is twice as hot as 10°C!

Complete Data Type Map

```
DATA
 │
 ├── CATEGORICAL (labels / groups)
 │     ├── Nominal   → No order   (colors, blood type, country)
 │     └── Ordinal   → Has order  (ratings, education level)
 │
 └── NUMERICAL (actual numbers)
       ├── Discrete  → Whole numbers only  (goals, students, coin flips)
       ├── Continuous→ Any value           (height, weight, temperature)
       ├── Interval  → Equal gaps, no true zero  (temperature °C, year)
       └── Ratio     → Equal gaps + true zero    (height cm, income)
```

Python — Identifying Data Types

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'name'       : ['Ali', 'Sana', 'Bilal'],      # Nominal categorical
    'grade'      : ['A', 'B', 'A'],                # Ordinal categorical
    'score'      : [85, 72, 90],                   # Discrete numerical
    'height_cm'  : [170.5, 162.3, 175.8],          # Continuous numerical
    'temp_c'     : [36.6, 37.1, 36.9],             # Interval
    'income_pkr' : [50000, 80000, 120000],         # Ratio
})
```

```
print(df.dtypes)          # shows int64, float64, object
print(df.describe())      # stats for numerical columns only
print(df.nunique())       # unique values per column

# Convert to ordered categorical:
df['grade'] = pd.Categorical(df['grade'], categories=['C','B','A'], ordered=True)
print(df['grade'] > 'B')  # True/False — ordering works now
```

2️⃣ Normal / Gaussian Distribution — The Most Important One

★ Why Normal Distribution is the KING of Statistics & ML

Most real-world measurements naturally follow this shape:
heights of people, exam scores, errors in measurements, weight, IQ scores.

Most ML algorithms ASSUME your data is normally distributed.
Linear Regression, Logistic Regression, PCA, Neural Network weight init —
ALL rely on this assumption.

Understanding Normal Distribution = Understanding the foundation of ML.

🔍 What Does It Look Like?

A perfect bell curve — symmetric, with most values clustered in the middle and fewer at the extremes.

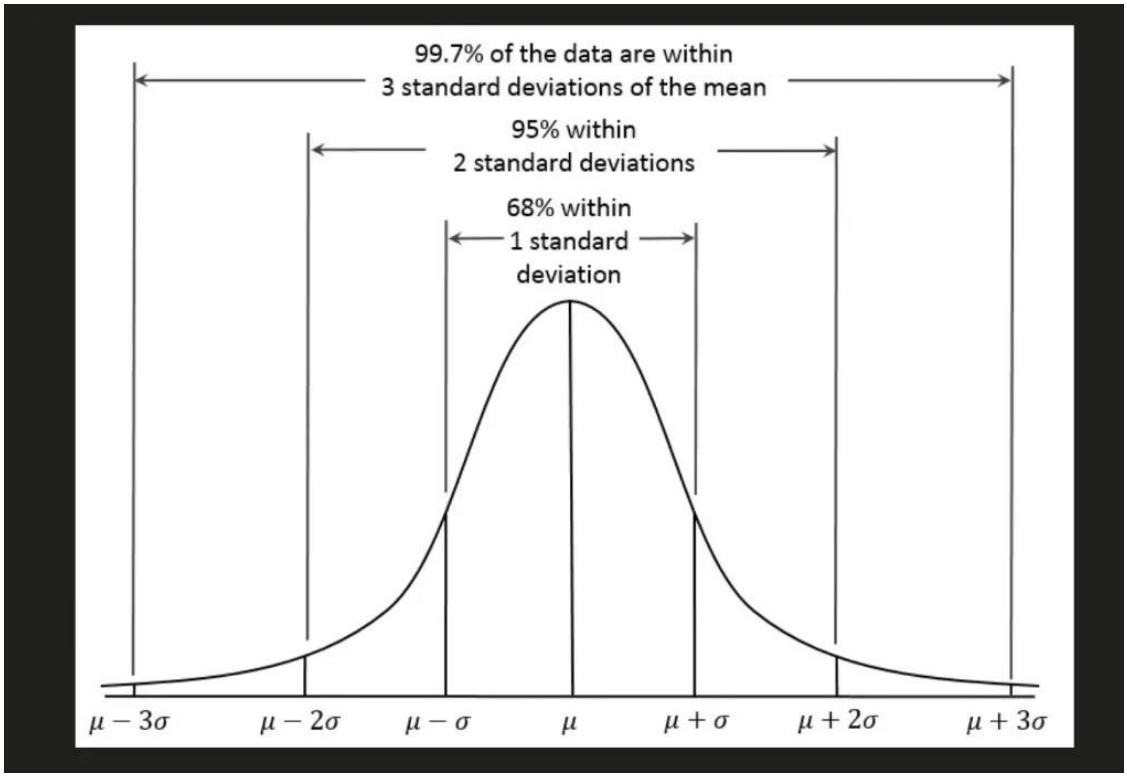

Figure 2: Normal Distribution — the Bell Curve showing μ ± 1σ, 2σ, 3σ

🧮 The Formula (You Don't Need to Memorise — Just Understand)

$$f(x) = \frac{1}{\sigma\sqrt{(2\pi)}} \times \exp\left(\frac{-(x-\mu)^2}{2\sigma^2}\right)$$

Where:

```
  μ (mu)     = mean    → controls the CENTER (where the peak is)
  σ (sigma)  = std dev → controls the WIDTH (how spread out it is)
  x          = any data point

Larger σ  → bell curve is WIDER and FLATTER
Smaller σ → bell curve is NARROWER and TALLER
```

The 68-95-99.7 Rule (Empirical Rule) — CRITICAL

This is one of the most used rules in all of statistics and ML. Memorise it.

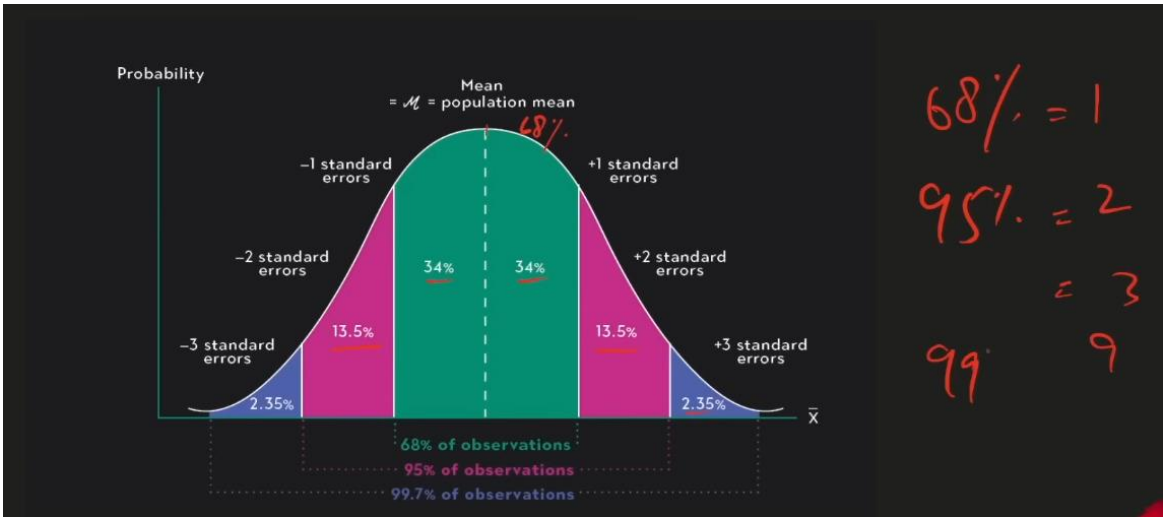

Figure 3: The Empirical Rule — 68%, 95%, 99.7% of data within 1, 2, 3 standard deviations

Range	% of Data	In your data	Real-life meaning
μ ± 1σ	68%	Between (mean − std) and (mean + std)	About 2 in 3 students fall within 1 std of the average
μ ± 2σ	95%	Between (mean − 2×std) and (mean + 2×std)	19 out of 20 values fall here — used for 95% confidence intervals
μ ± 3σ	99.7%	Between (mean − 3×std) and (mean + 3×std)	Almost everything. Values outside are extreme outliers (0.3%)

Real-Life Example — Exam Scores

Exam scores: Mean (μ) = 70, Std Dev (σ) = 10

68% of students scored between 70 − 10 = 60 and 70 + 10 = 80
95% of students scored between 70 − 20 = 50 and 70 + 20 = 90

99.7% of students scored between 70 − 30 = 40 and 70 + 30 = 100

If someone scores 95, they are more than 2σ above mean → top 2.5% of class.
This is how Z-scores work — measuring HOW MANY standard deviations from mean.

⍰ Why is Normal Distribution Important in Data Science & ML?

Where It Appears	Why It Matters
Central Limit Theorem	Sample means follow Normal distribution regardless of original data shape. This is WHY statistical tests work.
Linear Regression	Assumes errors (residuals) are normally distributed. If not, predictions are unreliable.
Neural Networks	Weights are initialized from Normal distribution. Random init avoids symmetry breaking problems.
Feature Scaling	StandardScaler assumes Normal dist. It shifts mean to 0 and std to 1. Only works well if data is Normal.
Confidence Intervals	The 95% CI formula uses ±1.96σ — directly from the Normal distribution's 95% rule.
Hypothesis Testing	Z-tests and t-tests assume Normal distribution of test statistics.

⍰ Python — Working with Normal Distribution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# ── Generate normally distributed data ──────────────────────────
mu    = 70    # mean
sigma = 10    # standard deviation
data  = np.random.normal(mu, sigma, 1000)   # 1000 random values

# ── Basic stats ─────────────────────────────────────────────────
print(f'Mean : {np.mean(data):.2f}')         # ≈ 70
print(f'Std  : {np.std(data, ddof=1):.2f}')  # ≈ 10

# ── 68-95-99.7 Rule check ───────────────────────────────────────
within_1sd = np.sum((data > mu-sigma)   & (data < mu+sigma))   / len(data)
within_2sd = np.sum((data > mu-2*sigma) & (data < mu+2*sigma)) / len(data)
within_3sd = np.sum((data > mu-3*sigma) & (data < mu+3*sigma)) / len(data)
print(f'Within 1σ: {within_1sd*100:.1f}%')   # ≈ 68%
print(f'Within 2σ: {within_2sd*100:.1f}%')   # ≈ 95%
print(f'Within 3σ: {within_3sd*100:.1f}%')   # ≈ 99.7%
```

```
# ── Plot the bell curve ──────────────────────────────────────
plt.figure(figsize=(10, 5))
plt.hist(data, bins=40, density=True, alpha=0.6, color='steelblue')
x = np.linspace(mu-4*sigma, mu+4*sigma, 300)
plt.plot(x, stats.norm.pdf(x, mu, sigma), 'r-', lw=2, label='Normal PDF')
plt.axvline(mu, color='black', linestyle='--', label=f'Mean={mu}')
plt.axvline(mu+sigma, color='orange', linestyle='--', label='±1σ')
plt.axvline(mu-sigma, color='orange', linestyle='--')
plt.legend()
plt.title('Normal Distribution — Bell Curve')
plt.show()
```

❓ How to Know if Your Data is Normally Distributed?

In real ML work you rarely have perfectly normal data. Here are the tests:

Method	What it does	Python
Histogram	Visual check — should look like a bell	plt.hist(data, bins='auto')
Q-Q Plot	Points should follow diagonal line if Normal	stats.probplot(data, plot=plt)
Shapiro-Wilk Test	p > 0.05 → data is Normal (best for small samples)	stats.shapiro(data)
K-S Test	Kolmogorov-Smirnov test for larger samples	stats.kstest(data, 'norm')
Skewness/Kurtosis	Skew≈0 and Kurtosis≈3 means Normal	stats.skew(data), stats.kurtosis(data)

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

data = np.random.normal(70, 10, 200)

# ── Method 1: Histogram ──────────────────────────────────────
plt.hist(data, bins=20)   # look for bell shape
plt.title('Is this Normal? — check the shape')
plt.show()

# ── Method 2: Q-Q Plot ──────────────────────────────────────
fig, ax = plt.subplots()
stats.probplot(data, dist='norm', plot=ax)
ax.set_title('Q-Q Plot — points on line = Normal')
plt.show()
```

```
# ── Method 3: Shapiro-Wilk Test ──────────────────────────────
stat, p_value = stats.shapiro(data)
print(f'Shapiro-Wilk: stat={stat:.4f}, p={p_value:.4f}')
if p_value > 0.05:
    print('✅ Data is likely NORMAL (fail to reject H0)')
else:
    print('❌ Data is NOT normal (reject H0)')

# ── Method 4: Skewness & Kurtosis ────────────────────────────
skew = stats.skew(data)
kurt = stats.kurtosis(data)   # excess kurtosis (Normal = 0)
print(f'Skewness  : {skew:.4f}  (should be near 0 for Normal)')
print(f'Kurtosis  : {kurt:.4f}  (should be near 0 for Normal)')
```

3️⃣ How to Normalize Data — Methods & Python Code

❓ What is Normalization / Standardization?

ML models get confused when features have very different scales.
Example: Age (20–80) vs Income (10,000–5,000,000) — income dominates by scale alone.

Normalization: rescales data to a standard range (usually 0–1).
Standardization: rescales data to have mean=0 and std=1.

These are the main tools to 'fix' your data before feeding it to ML models.

Method	Formula	Output Range	Best For
Min-Max Normalization	(x − min) / (max − min)	[0, 1]	When you know data has fixed bounds; Neural Networks
Z-score Standardization	(x − mean) / std	Unbounded, mean=0, std=1	When data is approximately Normal; most ML algorithms
Robust Scaling	(x − median) / IQR	Unbounded, outlier-resistant	When data has many outliers
Log Transform	log(x) or log(x+1)	Unbounded	Right-skewed data; income, prices
Box-Cox Transform	Finds best λ automatically	Near-Normal	Make non-normal data more Normal

❓ Python — All Normalization Methods

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
from scipy import stats

data = np.array([[10], [20], [30], [40], [500]])  # 500 is an outlier

# ── Method 1: Min-Max Normalization ──────────────────────────────
scaler_mm = MinMaxScaler()          # scales to [0, 1]
data_minmax = scaler_mm.fit_transform(data)
print('Min-Max:', data_minmax.flatten())
# Output: [0.    0.021 0.041 0.061 1.   ]  ← 500 becomes 1.0

# ── Method 2: Z-score Standardization ────────────────────────────
scaler_std = StandardScaler()       # mean=0, std=1
data_std = scaler_std.fit_transform(data)
```

```
print('Z-score:', data_std.flatten())

# ── Method 3: Robust Scaling (handles outliers) ──────────────────────
scaler_rob = RobustScaler()        # uses median & IQR
data_robust = scaler_rob.fit_transform(data)
print('Robust:', data_robust.flatten())
# Outlier 500 has much less influence here

# ── Method 4: Log Transform (manual) ─────────────────────────────────
data_log = np.log1p(data)          # log(x+1) to handle zeros
print('Log:', data_log.flatten())

# ── Method 5: Box-Cox (needs all positive values) ────────────────────
positive_data = np.array([10, 20, 30, 40, 100])
data_boxcox, best_lambda = stats.boxcox(positive_data)
print(f'Box-Cox lambda: {best_lambda:.3f}')
print('Box-Cox:', data_boxcox)
```

4 All Probability Distributions Explained

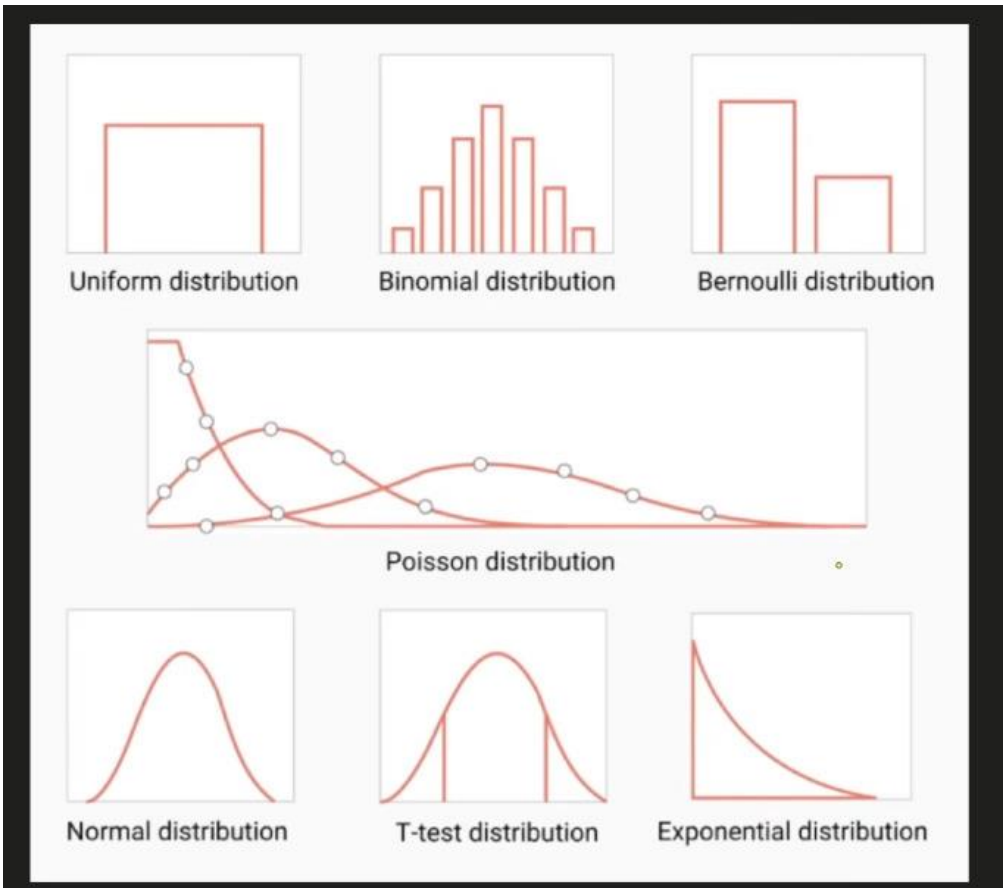

Figure 4: Types of Probability Distributions at a glance

Discrete vs Continuous Distributions

DISCRETE: Data can only take specific separate values (whole numbers).
Examples: Binomial, Bernoulli, Poisson, Geometric
Think: Number of goals, coin flip outcomes, number of emails per hour

CONTINUOUS: Data can take any value in a range (infinite possibilities).
Examples: Normal, Exponential, Uniform (continuous), T-distribution
Think: Height, weight, time, temperature

4.1 — Uniform Distribution

Every outcome has the SAME probability. Flat, equal chance for everything.

Real-Life Analogy

Rolling a fair die: 1, 2, 3, 4, 5, 6 — each has exactly 1/6 chance.
Picking a random number between 1 and 10 — every number equally likely.
Random number generator in computers — perfectly uniform.

Discrete Uniform	Continuous Uniform
Finite outcomes — each has equal probability Example: fair die (1-6)	Any value in [a, b] equally likely Example: random number in [0, 1]

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Discrete Uniform — rolling a die 1000 times
die_rolls = np.random.randint(1, 7, 1000)
plt.hist(die_rolls, bins=6, rwidth=0.8, color='steelblue')
plt.title('Discrete Uniform — Die Rolls')
plt.show()

# Continuous Uniform — between 0 and 10
data = np.random.uniform(0, 10, 1000)
plt.hist(data, bins=20, color='teal')
plt.title('Continuous Uniform Distribution')
plt.show()
```

4.2 — Bernoulli Distribution

The SIMPLEST distribution — only TWO possible outcomes: SUCCESS (1) or FAILURE (0).

One single trial. One event. One probability p.

Real-Life Analogy

Flipping one coin: Heads (1) or Tails (0).
One patient: Recovered (1) or Not recovered (0).
One email: Spam (1) or Not spam (0).
One click: Clicked (1) or Not clicked (0).

Bernoulli(p) — p is the probability of success (1).
If p=0.7 → 70% chance of success, 30% chance of failure.

```
from scipy import stats
import numpy as np

p = 0.7   # probability of success (heads)

# Single Bernoulli trial
outcome = stats.bernoulli.rvs(p)   # 0 or 1
print(f'Outcome: {outcome}')        # e.g., 1 (heads)

# 1000 coin flips
flips = stats.bernoulli.rvs(p, size=1000)
```

```
print(f'Heads: {flips.sum()}, Tails: {1000-flips.sum()}')

# In ML: this is exactly what Logistic Regression outputs!
# Output probability → threshold at 0.5 → Bernoulli outcome (0 or 1)
```

4.3 — Binomial Distribution

Binomial = MULTIPLE Bernoulli trials. How many successes in n trials?

Real-Life Analogy

Flip a coin 10 times. How many heads do you get?
That count (0 to 10) follows a Binomial distribution.

Binomial(n, p):
- n = number of trials
- p = probability of success in each trial
- X = number of successes (this is what we're finding)

Bernoulli = special case of Binomial where n=1.

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

n = 10    # 10 coin flips
p = 0.5   # fair coin

# Probability of getting exactly 6 heads
prob_6 = stats.binom.pmf(6, n, p)    # pmf = probability mass function
print(f'P(exactly 6 heads) = {prob_6:.4f}')   # 0.2051

# Probability of 6 or fewer heads
prob_leq6 = stats.binom.cdf(6, n, p)  # cdf = cumulative distribution
print(f'P(6 or fewer heads) = {prob_leq6:.4f}')  # 0.8281

# Simulate 1000 experiments of 10 flips each
results = stats.binom.rvs(n, p, size=1000)
plt.hist(results, bins=range(12), rwidth=0.8, color='steelblue')
plt.xlabel('Number of Heads')
plt.title('Binomial(n=10, p=0.5) — 1000 Simulations')
plt.show()
```

4.4 — Poisson Distribution

How many times does an event happen in a FIXED time period?

Used when events happen randomly but at a known average rate (λ, lambda).

Real-Life Analogy

You receive on average 5 emails per hour (λ = 5).
What is the probability you get exactly 3 emails in the next hour?

Other examples:
- Number of cars passing a toll booth per minute (λ = avg per minute)
- Number of goals in a football match (λ = avg goals per match)
- Number of server crashes per day (λ = avg crashes per day)

Key conditions: events are independent, constant average rate.

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

lam = 5   # average 5 emails per hour

# Probability of exactly 3 emails
prob_3 = stats.poisson.pmf(3, lam)
print(f'P(3 emails) = {prob_3:.4f}')   # 0.1404

# Probability of 8 or fewer emails
prob_leq8 = stats.poisson.cdf(8, lam)
print(f'P(≤8 emails) = {prob_leq8:.4f}')  # 0.9319

# Plot Poisson distribution for different lambdas
x = np.arange(0, 20)
for lam in [1, 4, 10]:
    plt.plot(x, stats.poisson.pmf(x, lam), marker='o', label=f'λ={lam}')
plt.title('Poisson Distribution — different λ values')
plt.legend()
plt.show()
```

4.5 — Normal / Gaussian Distribution (Continuous)

Already covered in detail in Section 2. Key reminder:

Quick Recap

Shape: Perfect bell curve, symmetric around mean (μ).
Parameters: μ (mean = center) and σ (std dev = width).
Rule: 68% within 1σ, 95% within 2σ, 99.7% within 3σ.
Special case: Standard Normal has μ=0, σ=1.
Z-score = (x − μ) / σ tells you how many std devs from the mean.

4.6 — T-Distribution (Student's t)

Similar to Normal distribution but with HEAVIER tails — used when sample size is SMALL.

Real-Life Analogy

You want to estimate the average height of all Pakistanis.
But you only measured 15 people (small sample).

With a small sample, you are LESS certain about the true mean.
The t-distribution accounts for this extra uncertainty — its tails are wider.

As sample size grows → t-distribution becomes Normal.
Rule of thumb: use t when n < 30.

Used in: t-tests (comparing means), confidence intervals with small samples.

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-4, 4, 300)

# Compare Normal vs t-distribution
plt.plot(x, stats.norm.pdf(x),  label='Normal (infinite df)', lw=2)
plt.plot(x, stats.t.pdf(x, df=3),  label='t (df=3)',  lw=2, linestyle='--')
plt.plot(x, stats.t.pdf(x, df=10), label='t (df=10)', lw=2, linestyle='-.')
plt.title('t-distribution vs Normal')
plt.legend()
plt.show()

# One-sample t-test: is sample mean significantly different from μ=70?
sample = np.random.normal(72, 10, 20)   # small sample (n=20)
t_stat, p_value = stats.ttest_1samp(sample, popmean=70)
print(f't-statistic: {t_stat:.4f}')
print(f'p-value    : {p_value:.4f}')
if p_value < 0.05:
    print('Significantly different from 70')
```

4.7 — Exponential Distribution

How long until the NEXT event happens? Time between events.

Continuous counterpart of Poisson. Memoryless — the past doesn't affect future.

Real-Life Analogy

You know customers arrive at 5 per hour on average (Poisson).
How long do you WAIT until the next customer? → Exponential!

More examples:
- How long until the next earthquake? (given average rate)
- How long until the next server request?

Battery lifetime — how long until failure?

Parameter: λ (rate). Mean waiting time = 1/λ.
If λ=5 (5 per hour), mean wait = 1/5 hour = 12 minutes.

```
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

lam = 5   # 5 customers per hour → scale = 1/lam = 12 minutes

# Probability of waiting more than 20 minutes (1/3 hour)
# P(X > 1/3) = 1 - CDF(1/3)
prob_gt20 = 1 - stats.expon.cdf(1/3, scale=1/lam)
print(f'P(wait > 20 min) = {prob_gt20:.4f}')   # 0.1889

# Simulate wait times
wait_times = stats.expon.rvs(scale=1/lam, size=1000)
plt.hist(wait_times, bins=40, color='salmon')
plt.xlabel('Wait Time (hours)')
plt.title('Exponential Distribution — Time Between Customers')
plt.show()
```

5 All Chart Types — Explained with Python

Why Different Charts?

Different data types and questions need different visualisations.
Using the wrong chart is like using a spoon to cut bread — technically possible but wrong!

Rule of thumb:
Want to see DISTRIBUTION of data? → Histogram or Box Plot
Want to COMPARE categories? → Bar Chart
Want to see RELATIONSHIP between two variables? → Scatter Plot
Want to see CHANGE over time? → Line Chart

Histogram — The Distribution Detective

Groups data into bins and shows how many values fall in each bin.

Best for: understanding the SHAPE and DISTRIBUTION of a single numerical variable.

Histogram vs Bar Chart — Key Difference

Histogram: X-axis is NUMERICAL (continuous). Bars touch each other (no gap).
Used for one numerical variable's distribution.

Bar Chart: X-axis is CATEGORICAL. Bars have gaps between them.
Used to compare values across different categories.

Easy trick: if there are gaps between bars → Bar Chart.
If bars are touching → Histogram.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

data = np.random.normal(70, 15, 500)   # exam scores

# ── Histogram (matplotlib) ──────────────────────────────
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.hist(data, bins=20, color='steelblue', edgecolor='white')
plt.title('Histogram — Exam Scores')
plt.xlabel('Score')
plt.ylabel('Frequency')

# ── Histogram with KDE (seaborn — much prettier) ────────
plt.subplot(1, 2, 2)
sns.histplot(data, kde=True, color='steelblue', bins=20)
plt.title('Histogram + KDE (seaborn)')
plt.tight_layout()
plt.show()
```

```
# When to use: understanding if data is Normal, skewed, bimodal etc.
# bins parameter: more bins = more detail, fewer bins = smoother view
```

Box Plot — The Outlier Detector

Shows the 5-number summary: Min, Q1, Median, Q3, Max — plus outliers as dots.

Best for: comparing distributions across groups and spotting outliers instantly.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

data = {
    'Math'    : np.random.normal(70, 10, 100),
    'Science' : np.random.normal(65, 15, 100),
    'English' : np.random.normal(75, 8, 100),
}
df = pd.DataFrame(data)

# ── Box plot (matplotlib) ──────────────────────────────────
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.boxplot(df.values, labels=df.columns)
plt.title('Box Plot — 3 Subjects')
plt.ylabel('Score')

# ── Box plot (seaborn — much easier) ──────────────────────
plt.subplot(1, 2, 2)
df_melt = df.melt(var_name='Subject', value_name='Score')
sns.boxplot(x='Subject', y='Score', data=df_melt)
plt.title('Box Plot (seaborn)')
plt.tight_layout()
plt.show()

# Reading a box plot:
#   Bottom line = Min (or lower whisker)
#   Box bottom  = Q1 (25th percentile)
#   Middle line = Median
#   Box top     = Q3 (75th percentile)
#   Top line    = Max (or upper whisker)
#   Dots        = Outliers
```

Bar Chart vs Bar Plot

Bar Chart: compares values across CATEGORIES. Bars are separate with gaps.

Bar Plot (in seaborn): same idea but can also show error bars (confidence intervals).

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import pandas as pd

subjects = ['Math', 'Science', 'English', 'History']
avg_score = [72, 68, 75, 60]
df = pd.DataFrame({'Subject': subjects, 'Average': avg_score})

# ── Bar Chart (matplotlib) ──────────────────────────────────
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.bar(subjects, avg_score, color=['steelblue','teal','salmon','gold'])
plt.title('Bar Chart — Average Scores')
plt.ylabel('Score')

# ── Bar Plot (seaborn) ──────────────────────────────────────
plt.subplot(1, 2, 2)
sns.barplot(x='Subject', y='Average', data=df, palette='Blues_d')
plt.title('Bar Plot (seaborn)')
plt.tight_layout()
plt.show()
```

📈 Line Chart — For Trends Over Time

Shows how a value CHANGES over time or a sequence. Best for time series data.

```
import matplotlib.pyplot as plt
import numpy as np

months = ['Jan','Feb','Mar','Apr','May','Jun']
sales  = [120, 135, 148, 162, 155, 178]

plt.figure(figsize=(8, 4))
plt.plot(months, sales, marker='o', color='steelblue', linewidth=2)
plt.fill_between(months, sales, alpha=0.2, color='steelblue')  # area under line
plt.title('Monthly Sales — Line Chart')
plt.ylabel('Sales (units)')
plt.grid(True, alpha=0.3)
plt.show()
```

🔵 Scatter Plot — Relationship Between Two Variables

Each dot = one data point (x, y). Shows if two variables are related (correlated).

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

height = np.random.normal(170, 10, 200)
weight = height * 0.4 + np.random.normal(0, 5, 200)  # correlated

# ── Scatter plot ────────────────────────────────────────────
```

```
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.scatter(height, weight, alpha=0.5, color='steelblue')
plt.xlabel('Height (cm)')
plt.ylabel('Weight (kg)')
plt.title('Scatter Plot — Height vs Weight')

# ── Scatter with regression line (seaborn) ──────────────────────
plt.subplot(1, 2, 2)
sns.regplot(x=height, y=weight, scatter_kws={'alpha':0.4})
plt.title('Scatter + Regression Line')
plt.tight_layout()
plt.show()
```

🔍 Other Important Charts

Chart	Best For	Python
Pie Chart	Show proportions/percentages of a whole	plt.pie(values, labels=labels)
Violin Plot	Like box plot but shows full distribution shape too	sns.violinplot(x, y, data=df)
Heatmap	Show correlation matrix or 2D data as colour grid	sns.heatmap(df.corr(), annot=True)
Pair Plot	All scatter plots between all variable pairs at once	sns.pairplot(df)
KDE Plot	Smooth version of histogram (probability density)	sns.kdeplot(data)
Q-Q Plot	Check if data is Normal	stats.probplot(data, plot=plt)
Count Plot	Count of each category (like bar for categoricals)	sns.countplot(x='col', data=df)

🔍 All Charts — Quick Decision Guide

```
What do you want to show?

Distribution of ONE numerical variable?
  → Histogram  (shape, skew, outliers)
  → KDE Plot   (smooth histogram)
  → Box Plot   (IQR, outliers, 5-number summary)
  → Violin     (box plot + density shape)
```

```
Compare VALUES across CATEGORIES?
  → Bar Chart  (single comparison)
  → Grouped Bar (multiple groups)

Show PROPORTIONS of a whole?
  → Pie Chart  (use sparingly — hard to read accurately)

Relationship between TWO numerical variables?
  → Scatter Plot
  → Scatter + Regression Line

CHANGE OVER TIME?
  → Line Chart

Correlation between MANY variables?
  → Heatmap (correlation matrix)
  → Pair Plot (all scatter plots at once)

Check if data is NORMAL?
  → Q-Q Plot
  → Histogram with KDE
```

6️⃣ Task Answers — Your Questions

Q1: Why is Normal Distribution Important in Data Science & Analytics?

> ⭐ **The Short Answer**
> Because MOST real-world data is approximately Normal, AND most ML algorithms are BUILT on assumptions that rely on it.
> Understanding Normal distribution = understanding why most of statistics works.

Reason	Explanation
1. Central Limit Theorem	No matter what shape your original data is, the MEAN of many samples follows Normal dist. This is WHY statistical tests work on any data.
2. Most real data is Normal	Heights, weights, measurement errors, IQ scores, blood pressure — all naturally Normal. Nature tends toward it.
3. ML Model Assumptions	Linear Regression assumes residuals are Normal. Logistic Regression, Naive Bayes, LDA — all use Normal assumptions internally.
4. Neural Network Weight Init	Weights are initialised from Normal(0, σ). This prevents vanishing/exploding gradients and makes training stable.
5. Feature Scaling	StandardScaler (Z-score) works BEST when data is Normal. It makes all features comparable in scale.
6. Confidence Intervals	The ±1.96σ formula for 95% CI comes directly from Normal distribution. Without it, interval estimation would not work.
7. Hypothesis Testing	Z-tests, t-tests, ANOVA — all assume Normal distribution of test statistics.

Q2: How to Normalize Data — All Methods

Method	Formula	Range	When to Use
Min-Max Normalization	$(x-\min)/(\max-\min)$	[0, 1]	Neural networks, known bounded data
Z-score / Standardization	$(x-\mu)/\sigma$	Any, mean=0, std=1	Most ML algorithms, normally distributed data

Robust Scaling	(x−median)/IQR	Unbounded	Data with many outliers
Log Transform	log(x) or log(x+1)	Unbounded	Right-skewed data (income, prices)
Sqrt Transform	√x	Unbounded	Moderately skewed data, count data
Box-Cox Transform	Searches for best λ	Near-Normal	When you don't know which transform to use
Yeo-Johnson	Like Box-Cox but allows zeros/negatives	Near-Normal	Box-Cox but data has 0 or negative values

⍰ Complete Normalization Python Code

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import (MinMaxScaler, StandardScaler,
                                   RobustScaler, PowerTransformer)
from scipy import stats
import matplotlib.pyplot as plt

# Sample skewed data (like income)
np.random.seed(42)
raw = np.random.exponential(scale=10, size=500)   # right-skewed
raw = raw.reshape(-1, 1)                           # reshape for sklearn

# ── Min-Max Normalization ──────────────────────────────────
mm = MinMaxScaler()
data_mm = mm.fit_transform(raw).flatten()

# ── Z-score Standardization ────────────────────────────────
std = StandardScaler()
data_std = std.fit_transform(raw).flatten()

# ── Robust Scaling ─────────────────────────────────────────
rob = RobustScaler()
data_rob = rob.fit_transform(raw).flatten()

# ── Log Transform ──────────────────────────────────────────
data_log = np.log1p(raw.flatten())   # log(x+1) handles zeros

# ── Box-Cox (sklearn's PowerTransformer) ───────────────────
pt_bc = PowerTransformer(method='box-cox')   # data must be positive
data_bc = pt_bc.fit_transform(raw).flatten()

# ── Yeo-Johnson (handles negatives too) ────────────────────
pt_yj = PowerTransformer(method='yeo-johnson')
data_yj = pt_yj.fit_transform(raw).flatten()
```

```
# ── Compare all transforms visually ──────────────────────────────
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
datasets = [('Original', raw.flatten()), ('Min-Max', data_mm),
            ('Z-score', data_std), ('Log', data_log),
            ('Box-Cox', data_bc), ('Yeo-Johnson', data_yj)]
for ax, (name, d) in zip(axes.flatten(), datasets):
    ax.hist(d, bins=30, color='steelblue', edgecolor='white')
    ax.set_title(name)
plt.suptitle('Effect of Different Normalization Methods')
plt.tight_layout()
plt.show()

# ── Check if result is Normal after transform ────────────────────
for name, d in [('Log', data_log), ('Box-Cox', data_bc)]:
    stat, p = stats.shapiro(d[:50])   # Shapiro needs small sample
    print(f'{name}: p={p:.4f} — {"Normal" if p>0.05 else "Not Normal"}')
```

⚡ Final Cheat Sheet — Everything at a Glance

Data Types

Type	Sub-type	Example	Python
Categorical	Nominal	Blood type, country	pd.Categorical / object
Categorical	Ordinal	Star ratings, grade	pd.Categorical(ordered=True)
Numerical	Discrete	Number of goals	int64
Numerical	Continuous	Height, weight	float64
Numerical	Interval	Temperature °C	float64
Numerical	Ratio	Income, height cm	float64

Distributions

Distribution	Type	Parameter	Real-World Use
Normal / Gaussian	Continuous	μ, σ	Heights, errors, ML weight init, most ML algorithms
Uniform	Both	a, b	Fair dice, random number generation
Bernoulli	Discrete	p	Single coin flip, spam/not spam, click/no click
Binomial	Discrete	n, p	n coin flips — how many heads?
Poisson	Discrete	λ (rate)	Events per time unit (emails/hr, goals/match)
Exponential	Continuous	λ	Time BETWEEN events (wait time, battery life)
T-distribution	Continuous	df (degrees)	Small sample testing (n<30)

Charts

Chart	Best For	Code

Histogram	Distribution of one numerical var	plt.hist(data) / sns.histplot(data, kde=True)
Box Plot	Outliers, IQR, compare groups	sns.boxplot(x, y, data=df)
Bar Chart	Compare categories	plt.bar(cats, vals) / sns.barplot()
Scatter Plot	Relationship between 2 variables	plt.scatter(x, y) / sns.regplot()
Line Chart	Change over time	plt.plot(x, y, marker='o')
Violin Plot	Distribution shape per group	sns.violinplot(x, y, data=df)
Heatmap	Correlations, 2D data	sns.heatmap(df.corr(), annot=True)
Q-Q Plot	Check if Normal	stats.probplot(data, plot=plt)